

Propozycja Tematu Pracy Magisterskiej: Wykorzystanie zk-Proof do Nowej Metody Autentykacji

Filip Gumuła

25 sierpnia 2025

Spis treści

1	Wprowadzenie	3
2	Teoretyczne Podstawy zk-Proof	4
2.1	Wprowadzenie do kryptografii	4
2.1.1	Kluczowe zasady kryptografii	4
2.1.2	Historia kryptografii	4
2.1.3	Kluczowe pojęcia w kryptografii	4
2.1.4	Zastosowania kryptografii	5
2.2	Zero-Knowledge Proofs: definicje i właściwości	5
2.2.1	Formalne właściwości ZKP	6
2.2.2	Intuicyjny przykład – Jaskinia Ali Baby	6
2.2.3	Rodzaje dowodów z wiedzą zerową	7
2.2.4	Zaufany Setup [5]	7
3	Projektowanie i implementacja systemu uwierzytelniania - analiza porównawcza	9
3.1	Założenia projektowe i wymagania systemu	9
3.1.1	Wymagania funkcjonalne	9
3.1.2	Wymagania нефункционалне	9
3.1.3	Architektura systemu	9
3.2	Metoda 1: Proste uwierzytelnianie tekstowe	10
3.2.1	Opis implementacji	10
3.2.2	Zalety prostego uwierzytelniania	10
3.2.3	Wady prostego uwierzytelniania	10
3.3	Metoda 2: Uwierzytelnianie z hashowaniem (Argon2)	11
3.3.1	Opis implementacji	11
3.3.2	Zalety hashowania Argon2	11
3.3.3	Wady hashowania Argon2	12
3.4	Metoda 3: Zero-Knowledge Authentication	12
3.4.1	Opis implementacji	12
3.4.2	Przepływ uwierzytelniania ZK	13
3.4.3	Kluczowe komponenty techniczne	14
3.4.4	Zalety Zero-Knowledge Authentication	15
3.4.5	Wady Zero-Knowledge Authentication	15
3.5	Analiza porównawcza	16
3.5.1	Porównanie bezpieczeństwa	16
3.5.2	Analiza wydajności	16
3.5.3	Złożoność implementacji	16
3.6	Wnioski projektowe	16
3.6.1	Rekomendacje zastosowania	16
3.6.2	Kierunki optymalizacji	17

1 Wprowadzenie

Celem pracy magisterskiej jest opracowanie nowej metody autentykacji użytkowników w systemach informatycznych z wykorzystaniem zero-knowledge proofs (zk-Proof), w szczególności SNARK oraz języka Noir. Tradycyjne metody autentykacji, oparte na przesyłaniu haseł lub innych danych wrażliwych do serwera, są podatne na liczne zagrożenia, takie jak przechwytywanie, ataki typu phishing czy naruszenia danych. Ta praca proponuje podejście, w którym użytkownik udowadnia swoją tożsamość za pomocą zk-Proof, bez konieczności ujawniania jakichkolwiek danych wrażliwych.

W ramach tej pracy szczególną uwagę poświęcono wykorzystaniu Noir jako języka do definiowania i generowania dowodów SNARK oraz jego integracji z Garaga do efektywnej weryfikacji proofów. Noir, jako dedykowane narzędzie dla zero-knowledge proofs, umożliwia wydajne oraz intuicyjne tworzenie systemów kryptograficznych, a jego kompatybilność z SNARK pozwala na skuteczne wdrożenie w praktycznych rozwiązaniach. Kluczowym celem jest zaprojektowanie oraz implementacja systemu autentykacji, który może być wykorzystywany w różnych środowiskach, takich jak zdecentralizowane aplikacje (dApps), systemy logowania bez hasła czy protokoły identyfikacji cyfrowej.

Podsumowując, praca ta koncentruje się na zastosowaniu SNARK w procesach autentykacji, przy jednoczesnym wykorzystaniu Noir do definiowania i generowania dowodów zk. Efektem końcowym będzie działający prototyp, który umożliwi użytkownikom bezpieczne uwierzytelnianie bez konieczności przesyłania haseł

2 Teoretyczne Podstawy zk-Proof

2.1 Wprowadzenie do kryptografii

Kryptografia to dziedzina zajmująca się metodami zabezpieczania informacji poprzez ich matematyczne przekształcenie. Jej celem jest uniemożliwienie dostępu do danych osobom nieuprawnionym, przy jednoczesnym zachowaniu ich dostępności dla osób posiadających odpowiednie uprawnienia. Początki kryptografii sięgają starożytności, kiedy stosowano podstawowe techniki szyfrowania oparte na zamianie lub przestawieniu znaków.

Współczesna kryptografia obejmuje szeroki zakres zagadnień, w tym szyfrowanie symetryczne i asymetryczne, funkcje skrótu oraz techniki bardziej zaawansowane, jak dowody wiedzy zerowej (Zero-Knowledge Proofs, zk-Proofs). Stosuje się ją nie tylko do ochrony poufnych danych, ale także do zapewnienia integralności i autentyczności informacji, co jest kluczowe w dzisiejszym cyfrowym świecie.

2.1.1 Kluczowe zasady kryptografii

Kryptografia opiera się na kilku podstawowych zasadach, które zapewniają bezpieczeństwo informacji:

- **Poufność (Confidentiality)** – tylko uprawnione osoby mogą odczytać zaszyfrowane dane.
- **Integralność (Integrity)** – zapewnia, że dane nie zostały zmienione lub sfalszowane w trakcie transmisji.
- **Autentyczność (Authenticity)** – pozwala zweryfikować, czy nadawca wiadomości jest tym, za kogo się podaje.
- **Niezaprzeczalność (Non-repudiation)** – zapobiega zaprzeczeniu przez nadawcę, że wysłał określoną wiadomość.

2.1.2 Historia kryptografii

Rzecz kryptografii można podzielić na kilka głównych etapów:

- **Kryptografia klasyczna** – obejmuje metody stosowane od starożytności do XX wieku. Przykładem jest szyfr Cezara, który polegał na przesunięciu każdej litery w alfabecie o ustaloną liczbę miejsc.
- **Kryptografia mechaniczna** – w XX wieku rozwinięto bardziej skomplikowane systemy szyfrowania, np. Enigmę, maszynę wirnikową używaną przez armię niemiecką w czasie II wojny światowej.
- **Kryptografia współczesna** – obecnie opiera się na zaawansowanych metodach matematycznych i obliczeniowych, w tym szyfrowaniu asymetrycznym oraz kryptografii postkwantowej.

2.1.3 Kluczowe pojęcia w kryptografii

Szyfrowanie i deszyfrowanie Szyfrowanie to proces przekształcania informacji w sposób, który czyni ją nieczytelną bez odpowiedniego klucza. Deszyfrowanie jest odwrotnym procesem, umożliwiającym odzyskanie oryginalnej treści.

Istnieją dwa główne typy szyfrowania:

- **Szyfrowanie symetryczne** – ten sam klucz służy zarówno do szyfrowania, jak i deszyfrowania (np. algorytm AES).
- **Szyfrowanie asymetryczne** – wykorzystuje parę kluczy: publiczny (do szyfrowania) i prywatny (do deszyfrowania). Przykładem jest algorytm RSA.

Funkcje skrótu Funkcje skrótu (np. SHA-256) przekształcają dowolne dane wejściowe w ciąg znaków o ustalonej długości, w sposób, który jest jednokierunkowy – tzn. nie da się łatwo odzyskać oryginalnej wartości na podstawie skrótu. Są one szeroko stosowane w kryptografii, np. do weryfikacji integralności danych.

Funkcje skrótu znajdują także zastosowanie w strukturach danych takich jak **drzewa Merkle**. Drzewa Merkle to hierarchiczne struktury, w których każdy węzeł jest funkcją skrótu swoich potomków. Zapewniają one integralność i spójność dużych zbiorów danych, umożliwiając efektywną i bezpieczną weryfikację poprawności danych w systemach rozproszonych, takich jak blockchain.

Zero-Knowledge Proofs (zk-Proofs) Jednym z bardziej zaawansowanych zastosowań kryptografii są dowody wiedzy zerowej. Pozwalają one jednej stronie (proverowi) wykazać, że posiada pewną informację, bez konieczności jej ujawniania. Technika ta znajduje zastosowanie m.in. w systemach prywatności blockchain i uwierzytelnianiu bez haseł.

2.1.4 Zastosowania kryptografii

Kryptografia znajduje zastosowanie w wielu dziedzinach technologii i bezpieczeństwa informacyjnego:

- **Bezpieczna komunikacja** – np. szyfrowanie transmisji internetowej za pomocą protokołów takich jak TLS/SSL (HTTPS).
- **Podpisy cyfrowe** – stosowane do zapewnienia autentyczności dokumentów elektronicznych.
- **Zabezpieczenia w bankowości** – np. w systemach 3D Secure oraz EMV chipach kart płatniczych.
- **Kryptografia w blockchain** – np. wykorzystanie funkcji skrótu i dowodów zk-Proofs do zapewnienia prywatności transakcji.
- **Kryptografia postkwantowa** – rozwijana w celu ochrony danych przed atakami komputerów kwantowych.

2.2 Zero-Knowledge Proofs: definicje i właściwości

Dowody z wiedzą zerową (ang. *Zero-Knowledge Proofs*, ZKP) to protokoły kryptograficzne, które umożliwiają jednej stronie (nazywanej *dowodzącym*) przekonanie innej strony (nazywanej *weryfikatorem*) o prawdziwości pewnego twierdzenia, nie ujawniając przy tym żadnych dodatkowych informacji poza faktem, że twierdzenie to jest prawdziwe [3].

2.2.1 Formalne właściwości ZKP

Aby protokół mógł być uznany za dowód z wiedzą zerową, musi spełniać trzy kluczowe właściwości [2]:

- **Kompletność (ang. *Completeness*)** – jeśli twierdzenie jest prawdziwe, uczciwy weryfikator zostanie przekonany przez uczciwego dowodzącego.
- **Poprawność (ang. *Soundness*)** – jeśli twierdzenie jest fałszywe, żaden nieuczciwy dowodzący nie zdoła przekonać uczciwego weryfikatora o jego prawdziwości.
- **Wiedza zerowa (ang. *Zero-Knowledge*)** – weryfikator nie uzyskuje żadnych informacji poza faktem, że twierdzenie jest prawdziwe.

2.2.2 Intuicyjny przykład – Jaskinia Ali Baby

Jednym z najbardziej znanych przykładów ilustrujących działanie dowodów z wiedzą zerową jest tzw. *Jaskinia Ali Baby* [1]. W tym scenariuszu występują dwie postacie: Tina (weryfikator) oraz Sam (dowodzący).

Podczas wspólnej wyprawy odkrywają oni jaskinię, w której znajdują się dwa wejścia prowadzące do dwóch różnych ścieżek – A i B. Wewnątrz znajduje się ukryte przejście zamknięte magicznymi drzwiami, które można otworzyć jedynie za pomocą tajnego hasła, znanego wyłącznie Samowi.

Celem eksperymentu jest przekonanie Tiny, że Sam faktycznie zna hasło otwierające drzwi, bez konieczności jego ujawniania. Aby tego dokonać, Sam wchodzi do jaskini, wybierając losową ścieżkę A lub B, natomiast Tina pozostaje na zewnątrz, nie wiedząc, którą drogą podążył. Następnie Tina losowo wskazuje, którą ścieżką Sam ma powrócić. Jeśli Sam wybrał ścieżkę A, ale Tina zażądała, aby wyszedł ścieżką B, jedynym sposobem na spełnienie tego warunku jest faktyczna znajomość hasła do drzwi.

Proces ten można powtórzyć wielokrotnie, aby zmniejszyć prawdopodobieństwo przypadkowego trafienia przez Sama na właściwą drogę. Po dostatecznej liczbie powtórzeń Tina nabiera wysokiego stopnia pewności, że Sam rzeczywiście zna hasło, nie mając jednak dostępu do samej jego treści. Ten mechanizm odzwierciedla sposób, w jaki działają dowody z wiedzą zerową w kryptografii – umożliwiają one weryfikację prawdziwości twierdzenia bez ujawniania żadnych dodatkowych informacji poza faktem, że twierdzenie to jest prawdziwe [1].

Jak przykład spełnia trzy cechy ZKP? Przykład Jaskini Ali Baby doskonale ilustruje trzy podstawowe właściwości dowodów wiedzy zerowej:

- **Kompletność (ang. *Completeness*)** – Jeśli Sam rzeczywiście zna hasło do drzwi, zawsze będzie w stanie przejść z jednej ścieżki na drugą i spełnić wymaganie Tiny. Oznacza to, że uczciwy dowodzący (Sam) zawsze przekona uczciwego weryfikatora (Tinę).
- **Poprawność (ang. *Soundness*)** – Jeśli Sam nie zna hasła, nie jest w stanie przejść na drugą stronę i wrócić zgodnie z żądaniem Tiny. Teoretycznie może próbować zgadywać, ale szansa, że za każdym razem zgadnie poprawnie, maleje wykładniczo z każdym kolejnym powtórzeniem eksperymentu. Ostatecznie fałszywy dowodzący nie będzie w stanie oszukać weryfikatora.
- **Wiedza zerowa (ang. *Zero-Knowledge*)** – W trakcie całego procesu Tina nie zdobywa żadnej dodatkowej informacji poza faktem, że Sam rzeczywiście zna hasło. Sam nie ujawnia treści hasła ani żadnych wskazówek, które mogłyby pomóc Tinie je odtworzyć.

W rzeczywistych zastosowaniach kryptograficznych ZKP działa na podobnej zasadzie, wykorzystując obliczeniowe schematy dowodzenia wiedzy bez ujawniania samego sekretu. Systemy te bazują na funkcjach matematycznych i kryptograficznych obwodach logicznych, w których dowodzący może wykazać znajomość pewnych wartości bez ich bezpośredniego ujawnienia [2].

2.2.3 Rodzaje dowodów z wiedzą zerową

Dowody z wiedzą zerową dzielą się na dwa główne typy [2] [4]:

- **Interaktywne dowody z wiedzą zerową** – Interaktywne dowody wiedzy zerowej (iZKP) to protokoły kryptograficzne, w których dowodzący (prover) i weryfikator (verifier) prowadzi wymianę komunikatów, aby udowodnić znajomość pewnej informacji bez jej ujawniania. W trakcie tej interakcji weryfikator zadaje pytania, a dowodzący musi na nie odpowiedzieć, dostarczając dowód znajomości sekretnej wartości. Jednym z najczęściej stosowanych interaktywnych protokołów ZKP jest schemat Fiat-Shamira. Polega on na tym, że weryfikator generuje losową wartość, którą wysyła jako wyzwanie do dowodzącego. Dowodzący następnie musi użyć swojej sekretnej wiedzy, aby wygenerować odpowiednią odpowiedź, którą weryfikator może sprawdzić. Jeśli odpowiedź jest poprawna, weryfikator akceptuje dowód i interakcja zostaje zakończona.
- **Nieinteraktywne dowody z wiedzą zerową (NIZK)** – Nieinteraktywne dowody wiedzy zerowej (niZKP) to protokoły, które nie wymagają interakcji pomiędzy dowodzącym a weryfikatorem. Zamiast tego, dowodzący generuje pojedynczy dowód, który może być zweryfikowany przez weryfikatora w dowolnym czasie, bez potrzeby dodatkowej komunikacji. Jednym z najczęściej stosowanych nieinteraktywnych protokołów ZKP jest SNARK (Succinct Non-Interactive Argument of Knowledge). SNARK wykorzystuje zaawansowane algorytmy matematyczne, aby wygenerować krótki, skondensowany dowód, który może być łatwo i szybko zweryfikowany.

2.2.4 Zaufany Setup [5]

Zaufany setup (ang. Trusted Setup) to proces inicjalizacyjny, w którym generowane są klucze kryptograficzne używane w niektórych systemach dowodów zerowej wiedzy (ZKP). Kluczowym celem tego procesu jest wygenerowanie zestawu parametrów, które umożliwią późniejsze tworzenie i weryfikację dowodów. Niektóre rodzaje ZKP, np. SNARKs, wymagają specjalnych parametrów kryptograficznych do tworzenia dowodów. Jeśli osoba lub organizacja, która generuje te parametry, zachowa ich pełną wersję, mogłaby stworzyć fałszywe dowody. Z tego powodu ważne jest, aby proces generowania kluczy był przeprowadzony w sposób bezpieczny i transparentny.

Jak działa Trusted Setup?

1. Wybór generatora – Proces zaczyna się od wyboru publicznych wartości matematycznych, takich jak generator grupy kryptograficznej.
2. Obliczanie parametrów – Tworzony jest zestaw parametrów publicznych oraz tajnych wartości, które umożliwiają generowanie dowodów.
3. Zniszczenie wartości tajnych – Aby system był bezpieczny, wartości tajne muszą zostać bezpowrotnie usunięte.

Jeśli osoba lub grupa, która przeprowadziła Trusted Setup, nie zniszczy tajnych wartości, mogłaby w przyszłości wygenerować fałszywe dowody, które wyglądałyby jak prawdziwe. To stanowi zagrożenie dla bezpieczeństwa systemu.

Jak można uniknąć problemu Trusted Setup?

- Multi-Party Computation (MPC) – zamiast jednej osoby, setup przeprowadza wiele niezależnych uczestników, a każdy generuje tylko część parametrów. Jeśli choć jedna osoba jest uczciwa, system pozostaje bezpieczny.
- Zk-STARKs zamiast zk-SNARKs – STARKs eliminują potrzebę Trusted Setup, wykorzystując inne mechanizmy matematyczne, np. funkcje mieszające zamiast krzywych eliptycznych.

Istnieje wiele implementacji dowodów z wiedzą zerową, z których każda ma swoje unikalne cechy, wady i zalety. Różnią się one m.in. rozmiarem dowodu, czasem generowania dowodu, czasem weryfikacji oraz wymaganym poziomem interakcji [2]. Do najważniejszych należą:

- **zk-SNARKs** (ang. *Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge*) – dowody, które są niewielkich rozmiarów i łatwe do weryfikacji. Generują kryptograficzny dowód przy użyciu krzywych eliptycznych, co jest bardziej efektywne pod względem zużycia gazu w porównaniu do metod opartych na funkcjach haszujących stosowanych w STARKs. Są wykorzystywane m.in. w kryptowalutach zapewniających prywatność, takich jak Zcash.
- **zk-STARKs** (ang. *Zero-Knowledge Scalable Transparent Arguments of Knowledge*) – eliminują potrzebę zaufanego setupu i są odporne na ataki kwantowe. STARKs wymagają minimalnej interakcji między dowodzącym a weryfikatorem, co czyni je szybszymi niż SNARKs. Dzięki temu są stosowane w skalowalnych rozwiązaniach dla blockchain, takich jak StarkNet.
- **PLONK** (ang. *Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge*) – wykorzystuje uniwersalny zaufany setup, który może być używany z dowolnym programem i obsługiwać dużą liczbę uczestników. PLONK jest bardziej elastyczny niż SNARKs, a jego popularność rośnie w systemach zk-rollups.
- **Bulletproofs** – krótkie, nieinteraktywne dowody zerowej wiedzy, które nie wymagają zaufanego setupu. Zostały zaprojektowane w celu umożliwienia prywatnych transakcji w kryptowalutach. Wykorzystywane są m.in. w Monero do ukrywania wartości transakcji.

Technologie te znajdują zastosowanie w wielu projektach zero-knowledge, takich jak StarkNet, ZKsync czy Loopring, które wdrażają rozwiązania zapewniające skalowalność i prywatność w blockchain [2].

Dowody z wiedzą zerową mają szerokie zastosowanie w różnych obszarach kryptografii:

- **Blockchain i kryptowaluty** – prywatne transakcje w Zcash (zk-SNARKs) oraz optymalizacja Layer 2 w Ethereum (zk-Rollups).
- **Uwierzytelnianie bez haseł** – systemy takie jak FIDO2 mogą korzystać z ZKP do zapewnienia tożsamości użytkownika bez ujawniania haseł.
- **Bezpieczne głosowanie elektroniczne** – systemy e-voting mogą wykorzystywać ZKP do weryfikacji głosów bez ujawniania ich treści.

3 Projektowanie i implementacja systemu uwierzytelniania - analiza porównawcza

3.1 Założenia projektowe i wymagania systemu

W ramach niniejszej pracy opracowano demonstracyjny system uwierzytelniania, który implementuje trzy różne podejścia do weryfikacji tożsamości użytkowników. Głównym celem było praktyczne porównanie tradycyjnych metod uwierzytelniania z nowatorskim podejściem wykorzystującym dowody wiedzy zerowej.

3.1.1 Wymagania funkcjonalne

System został zaprojektowany z myślą o realizacji następujących funkcjonalności:

- **Rejestracja użytkowników** – możliwość tworzenia nowych kont z wyborem metody uwierzytelniania
- **Logowanie** – weryfikacja tożsamości użytkowników przy użyciu wybranej metody
- **Bezpieczne przechowywanie danych** – odpowiednie zabezpieczenie informacji uwierzytelniających
- **Porównanie metod** – umożliwienie testowania różnych podejść w jednym środowisku

3.1.2 Wymagania niefunkcjonalne

Podczas projektowania systemu uwzględniono następujące aspekty:

- **Performance** – akceptowalny czas odpowiedzi dla użytkownika końcowego
- **Usability** – intuicyjny interfejs użytkownika niezależnie od wybranej metody
- **Skalowalność** – możliwość obsługi wielu użytkowników jednocześnie
- **Bezpieczeństwo** – odpowiedni poziom ochrony dla każdej zaimplementowanej metody

3.1.3 Architektura systemu

System został zbudowany w architekturze klient-serwer z następującymi komponentami:

- **Frontend** – aplikacja webowa napisana w Svelte, odpowiedzialna za interfejs użytkownika i generowanie dowodów kryptograficznych
- **Backend** – serwer Express.js w TypeScript, obsługujący logikę biznesową i weryfikację uwierzytelnień
- **Baza danych** – SurrealDB do przechowywania danych użytkowników i uwierzytelnień

[PLACEHOLDER: Screenshot - Strona główna aplikacji pokazująca wybór trzech metod uwierzytelniania]

3.2 Metoda 1: Proste uwierzytelnianie tekstowe

Pierwsza zaimplementowana metoda reprezentuje najbardziej elementarne i niewyrafinowane podejście do uwierzytelniania użytkowników, w którym hasła są przechowywane oraz weryfikowane w postaci jawnego, nieprzetworzonego tekstu. Ta metoda została świadomie zaprojektowana jako punkt odniesienia dla bardziej zaawansowanych rozwiązań, demonstrując jednocześnie fundamentalne problemy bezpieczeństwa związane z nieodpowiednim zarządzaniem danymi uwierzytelniającymi.

3.2.1 Opis implementacji

Architektura tej metody opiera się na niezwykle uproszczonym modelu przepływu danych, gdzie proces rejestracji nowego użytkownika sprowadza się do bezpośredniego, nieprzetworzonego zapisania podanego przez użytkownika hasła w strukturze bazy danych. Mechanizm ten charakteryzuje się całkowitym brakiem jakiegokolwiek transformacji kryptograficznej wprowadzanych danych, co oznacza, że hasło użytkownika zostaje zmagazynowane w identycznej postaci, w jakiej zostało wprowadzone przez użytkownika w interfejsie aplikacji.

Podczas procesu uwierzytelniania, system realizuje weryfikację tożsamości poprzez przeprowadzenie elementarnego porównania tekstowego pomiędzy hasłem wprowadzonym przez użytkownika a wartością przechowywaną w bazie danych. Ten proces weryfikacji nie wymaga żadnych operacji dekryptograficznych czy porównań hashowanych wartości, ponieważ wykorzystuje bezpośrednie porównanie ciągów znaków na poziomie aplikacji.

Warstwa serwerowa aplikacji eksponuje dwa fundamentalne endpointy RESTful API, które obsługują podstawowe operacje związane z zarządzaniem kontami użytkowników. Endpoint rejestracyjny przyjmuje dane użytkownika w formacie JSON, przeprowadza podstawową walidację poprawności formatowania wprowadzonych danych, a następnie zapisuje je bezpośrednio w bazie danych bez jakiegokolwiek obróbki kryptograficznej. Z kolei endpoint logowania porównuje otrzymane dane uwierzytelniające z zapisanymi wartościami, zwracając stosowną odpowiedź o powodzeniu lub niepowodzeniu procesu uwierzytelniania.

Należy podkreślić, że system ten został zaprojektowany wyłącznie w celach demonstracyjnych i edukacyjnych, służąc jako negatywny przykład implementacji uwierzytelniania. Metoda ta świadomie pomija wszelkie mechanizmy szyfrowania, hashowania czy inne zabezpieczenia kryptograficzne, co czyni ją całkowicie nieadekwatną do wykorzystania w jakimkolwiek środowisku produkcyjnym czy komercyjnym.

[PLACEHOLDER: Screenshot - Formularz prostej rejestracji i logowania]

3.2.2 Zalety prostego uwierzytelniania

- **Maksymalna prostota implementacji** – wymaga minimalnej ilości kodu i logiki
- **Zero narzutów obliczeniowych** – proces weryfikacji jest natychmiastowy
- **Brak dodatkowych zależności** – nie wymaga zewnętrznych bibliotek kryptograficznych
- **Łatwość debugowania** – wszystkie operacje są transparentne i łatwe do śledzenia

3.2.3 Wady prostego uwierzytelniania

- **Całkowity brak bezpieczeństwa** – hasła przechowywane w postaci jawnego tekstu
- **Podatność na wszystkie typy ataków** – man-in-the-middle, przejęcie bazy danych, shoulder surfing

- **Naruszenie standardów bezpieczeństwa** – sprzeczność z podstawowymi zasadami security by design
- **Problemy compliance** – niezgodność z regulacjami typu GDPR, SOX
- **Brak możliwości audytu** – niemożność śledzenia nieautoryzowanychostępów

3.3 Metoda 2: Uwierzytelnianie z hashowaniem (Argon2)

Druga metoda implementacyjna reprezentuje nowoczesny, przemysłany standard w dziedzinie bezpiecznego przechowywania oraz zarządzania hasłami użytkowników, wykorzystując zaawansowany algorytm hashowania Argon2 w połączeniu z techniką solenia kryptograficznego. To podejście stanowi znaczący krok naprzód w porównaniu z pierwszą metodą, wprowadzając fundamentalne mechanizmy bezpieczeństwa, które są powszechnie akceptowane i rekomendowane przez współczesne standardy cyberbezpieczeństwa.

3.3.1 Opis implementacji

Serce tej implementacji stanowi wysoce zaawansowana biblioteka Argon2, która została zaprojektowana specjalnie do generowania kryptograficznie bezpiecznych skrótów hasel podczas procesu rejestracji użytkowników. Argon2, będący zwycięzcą konkursu Password Hashing Competition, reprezentuje państwo sztuki w dziedzinie funkcji wyprowadzania kluczy, oferując niezrównaną odporność na współczesne zagrożenia kryptograficzne.

Proces transformacji hasła rozpoczyna się od generowania unikalnej, kryptograficznie silnej soli dla każdego indywidualnego użytkownika. Ta sól, będąca losową sekwencją bajtów o wysokiej entropii, zostaje następnie połączona z hasłem użytkownika w procesie hashowania, zapewniając tym samym, że nawet identyczne hasła różnych użytkowników generują całkowicie odmienne, niepowiązane ze sobą skróty kryptograficzne. Mechanizm ten skutecznie eliminuje możliwość wykorzystania ataków opartych na prekompilowanych tabelach tęczowych (rainbow tables).

Kluczowe komponenty architektoniczne tej implementacji obejmują szereg wyrafinowanych mechanizmów bezpieczeństwa. Generator soli kryptograficznej wykorzystuje wysokiej jakości źródła entropii systemu operacyjnego, tworząc dla każdego użytkownika unikalny, nieprzewidywalny identyfikator kryptograficzny o odpowiedniej długości. Mechanizm hashowania Argon2 został skonfigurowany z parametrami zapewniającymi odporność na ataki realizowane przy użyciu specjalistycznego sprzętu obliczeniowego, takiego jak procesory graficzne GPU czy dedykowane układy scalone ASIC.

Proces weryfikacji uwierzytelniania opiera się na sofistykowanym algorytmie porównania skrótów kryptograficznych, który pobiera hasło wprowadzone przez użytkownika, poddaje je identycznemu procesowi hashowania z wykorzystaniem oryginalnej soli, a następnie przeprowadza bezpieczne porównanie otrzymanego skrótu z wartością przechowywaną w bazie danych. System oferuje również możliwość konfiguracji parametrów kosztowych algorytmu, pozwalając na dostrojenie czasu wykonania operacji hashowania w zależności od wymagań bezpieczeństwa oraz dostępnych zasobów obliczeniowych.

[PLACEHOLDER: Screenshot - Formularz rejestracji z hashowaniem, być może pokazujący loading podczas hashowania]

3.3.2 Zalety hashowania Argon2

- **Branżowy standard** – powszechnie akceptowana metoda w systemach produkcyjnych

- **Ochrona przed atakami rainbow table** – unikalne sole uniemożliwiają użycie prekompilowanych tablic
- **Odporność na ataki brute-force** – konfigurowalny koszt obliczeniowy spowalnia ataki
- **Dojrzała technologia** – dostępność bibliotek i narzędzi we wszystkich językach programowania
- **Compliance** – zgodność z wymaganiami bezpieczeństwa i regulacjami prawnymi

3.3.3 Wady hashowania Argon2

- **Serwer nadal "zna" metodę weryfikacji** – pomimo hashowania, serwer posiada wszystkie dane potrzebne do weryfikacji hasła
- **Podatność na kompromitację serwera** – przejęcie bazy danych nadal stanowi zagrożenie
- **Scentralizowany model zaufania** – użytkownicy muszą zaufać, że serwer prawidłowo implementuje zabezpieczenia
- **Możliwość ataków offline** – po przejęciu hashów atakujący może przeprowadzać ataki bez ograniczeń czasowych
- **Ryzyko implementacyjne** – błędy w konfiguracji lub implementacji mogą osłabić bezpieczeństwo

[PLACEHOLDER: Screenshot - Porównanie czasów logowania dla różnych metod, możliwe wykresy]

3.4 Metoda 3: Zero-Knowledge Authentication

Trzecia i najbardziej wyrafinowana metoda implementacyjna stanowi prawdziwy przełom w dziedzinie uwierzytelniania użytkowników, wykorzystując zaawansowane dowody wiedzy zerowej (zero-knowledge proofs) do stworzenia systemu, w którym serwer może matematycznie zweryfikować autentyczność znajomości hasła przez użytkownika, nie uzyskując przy tym jakiegokolwiek informacji o samym hasle czy sposobie jego weryfikacji. Ta rewolucyjna technologia reprezentuje fundamentalną zmianę paradygmatu w podejściu do bezpieczeństwa systemów uwierzytelniania, eliminując tradycyjny model zaufania oparty na centralnym serwerze przechowującym dane uwierzytelniające.

3.4.1 Opis implementacji

Architektura systemu opiera się na starannie dobranym stosie technologicznym, który łączy najnowsze narzędzia i biblioteki kryptograficzne w spójną całość zdolną do realizacji dowodów wiedzy zerowej w środowisku przeglądarki internetowej. Każdy komponent tego ekosystemu pełni specjalizowaną funkcję w procesie generowania i weryfikacji dowodów kryptograficznych.

Fundamentem systemu jest Noir.js - nowoczesny, ekspresyjny język specjalistyczny przeznaczony do definiowania oraz implementacji obwodów kryptograficznych i generowania dowodów SNARK (Succinct Non-Interactive Arguments of Knowledge). Noir wyróżnia się intuicyjną składnią przypominającą języki wysokiego poziomu, jednocześnie oferując pełną kontrolę nad generowanymi obwodami kryptograficznymi. Język ten umożliwia programistom definiowanie złożonych

relacji matematycznych i logicznych, które następnie są kompilowane do wydajnych reprezentacji algebraicznych zdolnych do weryfikacji w systemach zero-knowledge.

Kluczową rolę w zapewnieniu spójności obliczeniowej między warstwą kliencką a serwerową pełni biblioteka Garaga, która dostarcza zaawansowaną implementację funkcji skrótu Poseidon. Ta kryptograficznie bezpieczna funkcja hashująca została specjalnie zaprojektowana do wykorzystania w systemach zero-knowledge, charakteryzując się optymalną wydajnością w środowisku obwodów algebraicznych. Garaga zapewnia identyczność obliczeniową funkcji Poseidon po obu stronach systemu - zarówno w środowisku JavaScript przeglądarki, jak i w serwerowym środowisku Node.js, co stanowi warunek konieczny dla poprawności generowanych i weryfikowanych dowodów.

System weryfikacji dowodów SNARK realizuje UltraHonk - zaawansowany protokół kryptograficzny należący do rodziny uniwersalnych systemów dowodzenia. UltraHonk charakteryzuje się szczególnie efektywną weryfikacją dowodów przy zachowaniu silnych gwarancji bezpieczeństwa, co czyni go idealnym wyborem dla aplikacji wymagających szybkiej weryfikacji dużej liczby dowodów. Protokół ten wykorzystuje nowoczesne techniki kryptograficzne oparte na krzywych eliptycznych, zapewniając jednocześnie kompaktowe rozmiary dowodów i wydajną weryfikację.

Całość infrastruktury kryptograficznej jest wspierana przez bibliotekę Barretenberg (BB.js), która stanowi kompleksowe narzędzie do obsługi wszystkich aspektów dowodów zero-knowledge w środowisku JavaScript. Barretenberg dostarcza niskopoziomowe implementacje algorytmów kryptograficznych, zarządzanie pamięcią obwodów algebraicznych, optymalizacje wydajnościowe oraz interfejsy do komunikacji z modułami WebAssembly zawierającymi krytyczne komponenty obliczeniowe.

3.4.2 Przepływ uwierzytelniania ZK

Proces uwierzytelniania opartego na dowodach wiedzy zerowej charakteryzuje się niezwykle złożonym, wieloetapowym przepływem danych i operacji kryptograficznych, który fundamentalnie różni się od tradycyjnych metod uwierzytelniania. Każdy etap tego procesu wymaga precyzyjnej koordynacji między warstwą kliencką a serwerową, jednocześnie zachowując fundamentalną właściwość zero-knowledge - serwer nigdy nie uzyskuje dostępu do informacji pozwalających na rekonstrukcję oryginalnego hasła użytkownika.

Proces rejestracji Procedura rejestracji nowego użytkownika w systemie ZK-Auth rozpoczyna się od wprowadzenia przez użytkownika podstawowych danych uwierzytelniających: nazwy użytkownika, hasła oraz dodatkowej soli kryptograficznej. Te dane wejściowe, wprowadzone w postaci zwykłego tekstu przez interfejs aplikacji webowej, stają się następnie przedmiotem serii skomplikowanych transformacji kryptograficznych realizowanych w pełni po stronie klienta.

Kluczowym momentem procesu rejestracji jest transformacja danych tekstowych do postaci BigInt przy wykorzystaniu funkcji Poseidon hash implementowanej przez bibliotekę Garaga. Ta operacja nie jest jedynie prostą konwersją formatu, lecz stanowi fundamentalny element architektury systemu, zapewniający matematyczną spójność między obliczeniami realizowanymi w przeglądarce a tymi wykonywanymi na serwerze. Funkcja Poseidon, będąca kryptograficznie silną funkcją skrótu zaprojektowaną specjalnie dla środowisk zero-knowledge, przekształca wprowadzone dane tekstowe w deterministyczne, ale nieprzewidywalne wartości numeryczne o wysokiej entropii.

Następnym krokiem jest kompilacja obwodu Noir oraz generacja kluczy weryfikacyjnych, co stanowi najbardziej obliczeniowo intensywny etap całego procesu. System inicjuje kompilator Noir, który analizuje zdefiniowany obwód kryptograficzny, optymalizuje jego strukturę algebraiczną, a następnie generuje pary kluczy: klucz prywatny służący do generowania dowodów oraz

klucz publiczny umożliwiającą ich weryfikację. Ten proces, realizowany w pełni w środowisku przeglądarki przy użyciu technologii WebAssembly, może wymagać kilku sekund intensywnych obliczeń, w zależności od złożoności obwodu oraz mocy obliczeniowej urządzenia klienta.

Po zakończeniu fazy przygotowawczej system przystępuje do generowania właściwego dowodu SNARK, który stanowi kryptograficzne potwierdzenie znajomości hasła bez jego ujawniania. Dowód ten zawiera matematyczny wniosek udowadniający, że użytkownik posiada wiedzę o wartościach wejściowych (hasło i sól), które po zastosowaniu funkcji Poseidon generują określony, publicznie znany skrót. Proces generowania dowodu wykorzystuje zaawansowane techniki kryptograficzne oparte na problemach dyskretnego logarytmu na krzywych eliptycznych, tworząc kompaktową reprezentację matematyczną tego dowodzenia.

Końcowym etapem procesu rejestracji jest transmisja wygenerowanego dowodu wraz z publicznymi parametrami wejściowymi do serwera aplikacji. Serwer, wykorzystując otrzymane klucze weryfikacyjne oraz bibliotekę UltraHonk, przeprowadza matematyczną weryfikację poprawności dowodu, nie uzyskując przy tym jakiegokolwiek informacji o oryginalnym hasle użytkownika. W przypadku pomyślnej weryfikacji, system zapisuje w bazie danych skrót kryptograficzny użytkownika oraz klucze weryfikacyjne, umożliwiając przyszłe procesy uwierzytelniania.

Proces logowania Procedura logowania w systemie ZK-Auth charakteryzuje się podobną złożonością do procesu rejestracji, jednak różni się kluczowymi detalami implementacyjnymi. Użytkownik wprowadza swoje dane uwierzytelniające w identycznej postaci jak podczas rejestracji, lecz tym razem system musi zweryfikować ich poprawność względem wcześniej zarejestrowanych danych, nie mając bezpośredniego dostępu do oryginalnych wartości.

Warstwa kliencka aplikacji, po otrzymaniu danych uwierzytelniających, inicjuje proces generowania nowego dowodu SNARK wykorzystując identyczne parametry i algorytmy jak podczas rejestracji. Kluczowym aspektem tego procesu jest deterministyczność obliczeń - identyczne dane wejściowe muszą generować identyczne dowody, co umożliwia serwerowi weryfikację autentyczności bez poznania oryginalnych danych. System ponownie wykorzystuje wcześniej skompilowany obwód Noir oraz zapisane klucze kryptograficzne, znacznie przyspieszając proces w porównaniu do pierwszej rejestracji.

Wygenerowany dowód, wraz z towarzyszącymi mu publicznymi parametrami, zostaje przesłany do serwera w bezpieczny sposób. Serwer aplikacji, mając dostęp do wcześniej zapisanych kluczy weryfikacyjnych oraz skrótu kryptograficznego użytkownika, przeprowadza proces weryfikacji dowodu przy użyciu protokołu UltraHonk. Weryfikacja ta potwierdza matematycznie, że przedłożony dowód został wygenerowany przez osobę posiadającą wiedzę o danych, które po zastosowaniu funkcji Poseidon generują oczekiwany skrót kryptograficzny.

W przypadku pomyślnej weryfikacji dowodu, serwer autoryzuje użytkownika i ustanawia bezpieczną sesję aplikacyjną. Cały proces logowania realizuje w ten sposób fundamentalną właściwość zero-knowledge: serwer uzyskuje matematyczną pewność co do autentyczności użytkownika, nie poznając przy tym żadnych informacji o jego hasle czy mechanizmie jego weryfikacji.

[PLACEHOLDER: Diagram - Schemat przepływu danych w procesie ZK authentication]

3.4.3 Kluczowe komponenty techniczne

Funkcja Poseidon Hash System wykorzystuje funkcję Poseidon hash zarówno w frontend'ie jak i backend'ie, zapewniając matematyczną spójność obliczeń. Garaga umożliwia identyczne obliczenia po obu stronach systemu, co jest kluczowe dla poprawności dowodów.

Obwód Noir Zdefiniowany obwód kryptograficzny implementuje logikę dowodzenia znajomości hasła poprzez:

- Przyjęcie prywatnych inputów (hasło, sól)
- Obliczenie Poseidon hash z tych wartości
- Porównanie z oczekiwanym hashem (publiczny input)
- Wygenerowanie dowodu poprawności bez ujawniania prywatnych danych

[PLACEHOLDER: Screenshot - Interfejs ZK rejestracji z loadingiem podczas generowania proof]

3.4.4 Zalety Zero-Knowledge Authentication

- **Właściwość zero-knowledge** – serwer nigdy nie poznaje ani nie przetwarza rzeczywistego hasła użytkownika
- **Kryptograficznie udowodnione bezpieczeństwo** – oparte na matematycznych gwarancjach zamiast założeń zaufania
- **Odporność na kompromitację serwera** – nawet przejęcie pełnych danych serwera nie ujawnia haseł użytkowników
- **Eliminacja scentralizowanego zaufania** – bezpieczeństwo oparte na matematyce, nie na zaufaniu do implementacji serwera
- **Nieemożność przeprowadzenia ataków offline** – brak danych pozwalających na ataki brute-force po stronie atakującego
- **Przyszłościowość** – technologia rozwijana jako standard dla prywatnych systemów uwierzytelniania

3.4.5 Wady Zero-Knowledge Authentication

- **Złożoność obliczeniowa** – generowanie dowodu SNARK wymaga znaczących zasobów obliczeniowych (kilka sekund)
- **Duży rozmiar dowodu** – wygenerowane dowody mogą osiągać rozmiary rzędu dziesiątek megabajtów
- **Kompleksowość techniczna implementacji** – wymaga głębokiej znajomości kryptografii i matematyki
- **Wymagania kompatybilności** – potrzeba nowoczesnych przeglądarek obsługujących WebAssembly
- **Czas weryfikacji** – choć krótszy niż generowanie, nadal znacząco dłuższy od tradycyjnych metod
- **Ograniczona dostępność narzędzi** – ekosystem bibliotek i frameworków nadal się rozwija

[PLACEHOLDER: Screenshot - Porównanie czasów wykonania wszystkich trzech metod w formie tabeli lub wykresu]

3.5 Analiza porównawcza

W celu kompleksowego porównania zaimplementowanych metod uwierzytelniania przeprowadzono analizę w następujących wymiarach:

3.5.1 Porównanie bezpieczeństwa

Aspekt bezpieczeństwa	Proste	Argon2	ZK-Auth
Ochrona przed MITM	Nie	Częściowa	Tak
Odporność na server breach	Nie	Ograniczona	Tak
Ataki brute-force	Podatne	Odporne	Odporne
Znajomość hasła przez serwer	Tak	Możliwa	Nie
Matematyczne gwarancje	Nie	Częściowe	Tak

Tabela 1: Porównanie aspektów bezpieczeństwa metod uwierzytelniania

3.5.2 Analiza wydajności

Na podstawie przeprowadzonych testów wydajnościowych zmierzono następujące parametry:

Metryka	Proste	Argon2	ZK-Auth
Czas rejestracji	<10ms	500ms	8-12s
Czas logowania	<5ms	500ms	8-12s
Rozmiar requestu	100B	200B	50MB
Zużycie CPU (client)	Minimalne	Minimalne	Wysokie
Zużycie CPU (server)	Minimalne	Średnie	Średnie

Tabela 2: Porównanie wydajności metod uwierzytelniania

[PLACEHOLDER: Wykres słupkowy pokazujący czasy odpowiedzi dla każdej metody]

3.5.3 Złożoność implementacji

Analiza złożoności implementacyjnej pod kątem:

- **Liczba linii kodu** – ZK-Auth wymaga 3x więcej kodu niż Argon2
- **Liczba zależności** – ZK-Auth wprowadza 8 dodatkowych bibliotek kryptograficznych
- **Czas implementacji** – szacowany stosunek 1:3:12 dla prostej:Argon2:ZK
- **Wymagane kompetencje** – ZK-Auth wymaga znajomości kryptografii i matematyki dyskretnej

3.6 Wnioski projektowe

3.6.1 Rekomendacje zastosowania

Na podstawie przeprowadzonej analizy można sformułować następujące rekomendacje:

Proste uwierzytelnianie Metoda ta nie powinna być stosowana w żadnych systemach produkcyjnych. Może służyć wyłącznie jako:

- Prototyp w fazie wczesnego rozwoju
- Narzędzie edukacyjne do demonstracji problemów bezpieczeństwa
- Baseline do porównań wydajnościowych

Hashowanie Argon2 Pozostaje gold standardem dla większości aplikacji komercyjnych w scenariuszach:

- Systemy wymagające szybkiego czasu odpowiedzi
- Aplikacje z ograniczonymi zasobami obliczeniowymi
- Środowiska gdzie ZK-Proof nie jest wspierane
- Tradycyjne aplikacje webowe i mobilne

Zero-Knowledge Authentication Rekomendowane w sytuacjach wysokich wymagań bezpieczeństwa:

- Aplikacje finansowe i bankowe
- Systemy przechowujące wrażliwe dane osobowe
- Środowiska o wysokim ryzyku kompromitacji serwera
- Aplikacje zdecentralizowane (dApps)
- Systemy gdzie prywatność jest kluczowa

[PLACEHOLDER: Diagram decision tree - Kiedy używać której metody uwierzytelniania]

3.6.2 Kierunki optymalizacji

Zidentyfikowano następujące obszary do dalszego rozwoju:

- **Optymalizacja rozmiaru dowodu** – implementacja technik kompresji i agregacji
- **Przyspieszenie generowania** – wykorzystanie acceleracji sprzętowej (GPU, WASM)
- **Poprawa UX** – implementacja progresywnego ładowania i feedback’u użytkownika
- **Standardizacja protokołu** – wypracowanie standardów interoperacyjności

Implementacja trzech metod uwierzytelniania wykazała, że choć ZK-Authentication wprowadza znaczące narzuty obliczeniowe, oferuje bezprecedensowy poziom bezpieczeństwa i prywatności. W miarę rozwoju technologii i poprawy wydajności, może stać się standardem dla aplikacji wymagających najwyższego poziomu bezpieczeństwa.

Literatura

- [1] 101 Blockchains. Zero-knowledge proof example – the ali baba cave, 2023. Dostęp: 01-02-2025.
- [2] Chainlink. Zero-knowledge proof (zkp) — explained, 2023. Dostęp: 01-02-2025.
- [3] Marcin Karbowski. *Podstawy kryptografii*. Helion, 2015.
- [4] NFTing. Zero-knowledge proof: Interactive vs. non-interactive, 2023. Dostęp: 01-02-2025.
- [5] Panther Team. Understanding trusted setups: A guide, 2022. Dostęp: 01-02-2025.